1.0 Data Processing For Machine Learning

To overcome the issues encountered during the data exploration and to ensure integrity, reliability, and usability of data for analysis and for machine learning processes it was necessary to deal with outliers and incorrect values. The most expensive cars present in the dataset, while appearing ludicrously priced were confirmed to be appropriate given the model of the car. While not erroneous, they are not representative of the bulk of the dataset, being out of consideration for the vast majority of consumers exemplified by the little representation within the data. On the other hand, extremely cheap cars will hinder the accuracy of the analysis as there is likely be an unmeasured variable rationalising the low price. This may be unknown damage unspecified in the dataset, for example a car having cat s/n write off status, an illegal marker or the seller simply wanting to sell swiftly thus presenting as a bargain. It is therefore safe to conclude that the integrity of these values is questionable as an accurate representation of price given the parameters captured in the dataset. However, given the prevalence of the cheap car market i.e. for first time drivers who are likely to damage a car, it was important that the data captures this segment of the market.

The strategy of eliminating both the top 2.5% and bottom 2.5% of cars with respect to price was implemented having been classified as outliers, narrowing the data to be examined within the price range of £1695.0 to £59995.0. Similar steps were applied with the mileage, instead using a one-sided approach to remove the upper 0.5% of values. Cars typically become more uneconomical to repair upon approaching 200,000 miles, so the merchantable quality of cars when approaching this figure, regardless of potential other enticing features (particularly price) reduces significantly. Meanwhile, cars with very low mileage simply may not have been driven so it would be inappropriate to treat them as extremes of values and therefore the data for subsequent analysis was narrowed to between 0-150,000 miles.

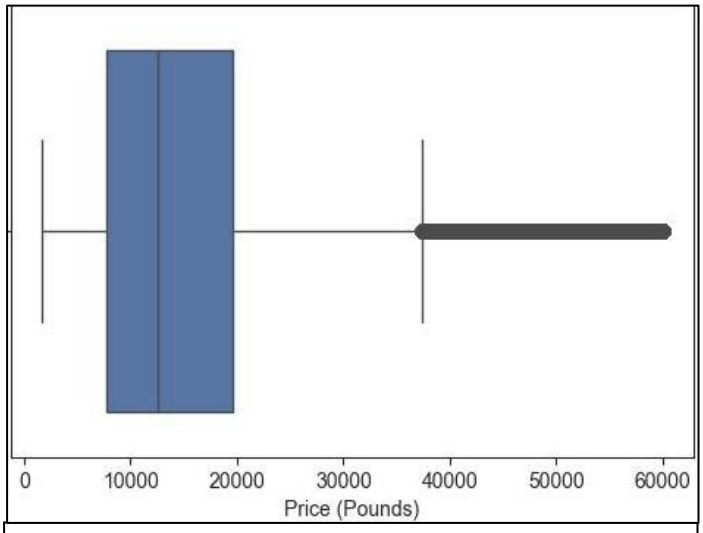

Figure 1 - Boxplot of Price Once Filtered

Inaccurate and unexpected values, such as cars listed as having been registered in the year 999, were discovered within the year of registration column. Once confirmed these were not more primitive forms of transport and consequently determined to be erroneous (assumably down to input error), it was necessary to remove these incorrect values. Upon further investigation, there were fewer than 30 instances per year where the registration date was pre-1986. Therefore, 1986 was determined as the oldest year to be included for subsequent analysis.

It was necessary to handle missing values as their inclusion can introduce bias, affect model performance, and potentially lead to inaccurate predictions, but simply dropping rows or columns with missing data results in the loss of valuable information which may skew the broader analysis. To mitigate this impact, late stage missing value imputation was instituted within the machine learning pipeline (***Figure 6, Figure 9)***. To further enhance the performance of the initial iteration, KNN imputation where k=2 was introduced for numeric features, which was found to be the optimal number of neighbours through experimentation. As a result of the implementation of KNN imputation more accurate replacements for missing values were provided thereby refining the dataset and improving subsequent modelling. To ensure data leakage does not occur, it was paramount the imputation occurred after the train/test split. For categorical features (***Figure 4***), missing values will be replaced with the most frequent value in each column.

2.0 Feature engineering

The characteristics of the dataset and the high cardinality of features, specifically standard make and standard model, made it appropriate to apply target encoding to the categorical features to be processed by the machine learning algorithms. Unlike One-hot encoding, target encoding preserves the original information without

creating a high-dimensional space by populating the row with the mean of the target for that specific category. This allowed for the machine learning algorithms to understand the categorical data. The typecasting of vehicle condition to a numeric data type was performed, specifically to binary form with 0 and 1 representing used and new cars respectively to numerically assess the impact as a feature. It was also possible to derive new informative features from those already present. The latest vehicle registration date was from 2020, so under the context that this is the most recent date in which the data was collected, recorded and assembled, the features "Age" and "Milesperyear"(***Figure 2)*** were produced, providing the age of the car and average number of miles driven per year per car respectively. Milesperyear provides information as to the driving behaviours and how frequently and extensively a vehicle has been driven, as well as an indication towards the overall picture of the vehicle's utilisation, whether that be more leisurely/recreational use or regular lengthy commutes.

A custom class named "GroupInfrequentCategories" was developed to address infrequent categories within features 'Make' and 'Model'. Instances where categories contain fewer than two occurrences were consolidated into a unified category labelled 'Other' within the respective features. This step led to performance enhancements in all models except for the linear regressor as it inherently assumes linear relationships between features and target and therefore the reduction of feature variability resulted in no improvement in performance.

```
[63]: machine['Milesperyear'] = machine['Mileage'] / (2020 - machine['Year of Registration']).replace(0, 1)

[64]: machine['Age'] = (2020 - machine['Year of Registration']).replace(0, 1)
```

Figure 2 – Code Implemented to Engineer New Features "Milesperyear" and "Age"

Previous machine learning iterations revealed the presence of non-linear relationships of the features with the target, so to improve competitiveness with models inherently adept at capturing such relationships, polynomial and interaction features were introduced from the original feature set, specifically tailored for the LASSO regressor. Iteratively, the optimum degree of non-linearity was determined by incrementally increasing the degree of polynomial features and assessing the trade-off between increasing model complexity and performance. Ultimately, polynomial features with a degree of 3 were deemed optimal and incorporated to effectively capture the dataset's underlying non-linear relationships, As increasing the degree of non-linearity beyond this point only negligibly improved the model's performance, the returns from added complexity were determined to be diminishing. Incorporating polynomial and interaction features should allow for the model to become more flexible at capturing the complex nature of the dataset, with the aim of achieving significant improvements in overall performance. With a degree value of 3 and 10 original features, the LASSO regressor introduced 51 new interaction and non-linear features into the dataset, which were comprised of combinations of the manually selected features. In total, there were 61 features available for use for the LASSO regressor.

To determine the average price per car-make depending on the condition of the vehicle, cars of the same make and condition were grouped together and introduced into the dataset (***Figure 3***). The identical procedure was followed, except the equivalent columns were produced using the model of the car. Comparing and contrasting the new and used car markets, and ultimately increasing the level of granularity from make to model will provide us with useful information regarding value retention between respective categories. Note. Respective columns won't be populated for categories that don't contain any instances of either new or used data.

```
adv["AvgMakeUsed"] = adv[adv['Vehicle condition'] == 0].groupby(['Make'])['Price'].transform('mean')

adv["AvgMakeUsed"] = adv["AvgMakeUsed"].fillna(adv.groupby("Make")['AvgMakeUsed'].transform("mean"))
```

Figure 3 – Code Implemented to Engineer New Feature "AvgMakelUsed" (Average Model Price Used).

3.0 Feature Selection and Dimensionality Reduction

While information regarding the specific part of the year in which the car was registered is provided within reg_code, extracting this information to produce a new column was deemed to be unrewarding given the scope of the investigation. Once this had been established, the features reg code and year of registration effectively inferred the same information, revealing a concern of multicollinearity. In combination with the fact that both of these features have the largest amount of missing data (7.92% , 8.29%) in comparison with others, the feature reg code was removed from the dataset. Though similar considerations were taken with the feature Age, its inclusion improved models without observing overfitting so its exclusion was not necessary. Crossover car and

van is a Boolean column, expressing whether or not the instance satisfies this condition (1) or not (0). This feature is describing the shape and structure of a car and therefore infers very similar information to body type despite those satisfying the condition also possessing a specified body type. It would be more appropriate to make this a category within body type, but due to the concern of multicollinearity this feature was also eliminated from the dataset. Public reference, the unique identifier in the dataset, was also dropped as its inclusion resulted in the breakdown of model performance.

```
target = 'Price'
cat_feat = ['Colour', 'Make', 'Model', 'Bodytype','Fueltype']
num_feat = ['Year of Registration', 'Mileage', 'Vehicle condition', 'Milesperyear', 'Age']
```

Figure 4 – Shared Features Manually Selected For Use Initially Before Application Of PCA/Feature Selection Methods

RFECV and Sequential Feature Selection (SFS) are methods for feature selection, with RFECV iteratively pruning less relevant features while evaluating model performance through cross-validation, and SFS iteratively selects features based on their individual contributions to model performance, which are determined by comparing the model's performance with and without each feature. These methods were explored alongside SelectKBest, aiming to produce optimum performing models. Principal Component Analysis (PCA) is a technique used to uncover underlying patterns in data by reducing computational complexity via dimension reduction, with an aim enhance the performance of machine learning algorithms. By retaining only the principal components that explain most of the variance(which indicates the ability to explain model outcome), PCA effectively removes noise and redundant information from the dataset, compressing the data into a lower-dimensional space without losing critical information.

All possible combinations of feature selection/dimension reduction methods mentioned were investigated: models using only manually selected features *(Figure 4)* with and without the implementation of feature selection methods and PCA, and models with polynomial features introduced, with and without the implementation of feature selection methods and PCA. Furthermore, PCA with feature selection methods were also investigated, effectively performing an exhaustive search for each model and the optimum combination to accompany the respective algorithms was therefore determined.

Note: specific details on configuration is outlined in *4.0 model building*. From ***Figure 5,*** approximately 95% of the variance for the LASSO regressor was captured at PCA=4, however, optimum model performance, determined iteratively, was achieved at PCA=15. This can be attributed to the fact that a greater number of principal components has a greater ability to capture non-linear relationships despite the negligible increase in variance in combination with the reduction of noise, expected when going from 61 features to 15. Therefore, PCA=15 was chosen because it provides the model with a richer representation of the underlying data structure, allowing it to capture more nuanced patterns and ultimately leading to improved performance. The XGB model also made use of PCA, and the similarity in the number of features between the original feature set (10 features) and the reduced feature set obtained through PCA (10 components) suggests that PCA did not enhance XGBoost by dimension reduction. Instead, it likely improved XGBoost's performance by imparting more favourable properties for the algorithm, such as better separability between classes or clearer decision boundaries.

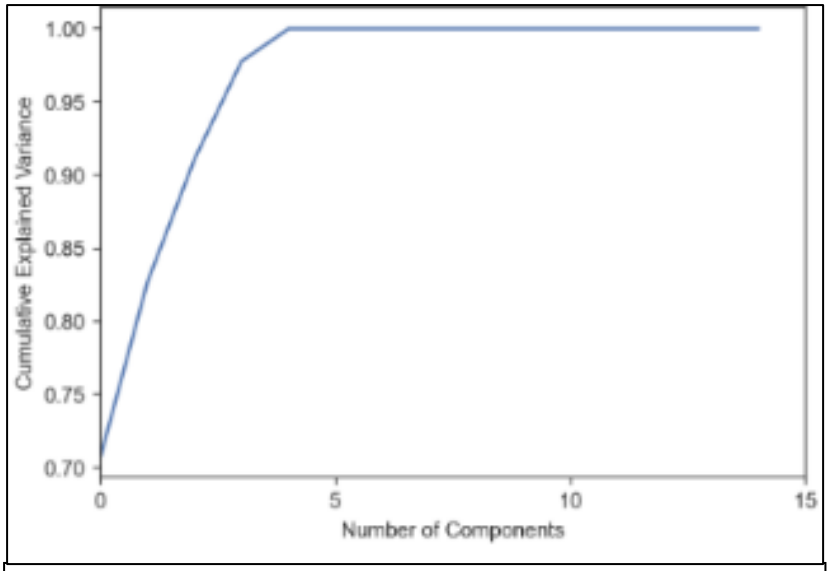

Figure 5 – Cumulative Explained Variance vs Number of Components For LASSO Regressor

Selectkbest was not competitive with other feature selection methods, as it may select redundant or irrelevant features if they have high individual scores but do not contribute much to the model when combined with other features. On the other hand, RFECV eliminates features iteratively, considering their collective impact on model performance, while SFS evaluates subsets of features to select those that collectively improve model performance. These approaches ensures that the synergistic contribution of features is assessed, not only individual influence.

4.0 Model building

The categorical and numeric features were defined independently due to the disparities in the requirements for data processing. With the imputation procedures outlined in *1. Data Processing For Machine learning* applied, the normalisation of numeric features was required to make the regression models more robust to outliers by ensuring their influence isn't disproportionately large. This was done via the MinMaxScaler (***Figure 6***), identifying the minimum and maximum values of the respective features and transforming them to 1 and 0 with every other data point being scaled proportionally. The target encoder and "Groupinfrequentcategories" class as described in *2.0 Feature Engineering* were then applied to the categorical features to be used for subsequent investigations.

Cross validation was performed to obtain a more robust assessment on the performance of the models and for the purpose of identifying potential issues such as under/overfitting and anomalous scores (i.e. because of the specific distribution of the test set). Hyperparameter optimisation is made possible by grid search, which yields model scores corresponding to the range of predefined hyperparameter values. Both grid search and cross validation are complimentary and were used together to find and validate the best possible hyperparameters available to each model.

4.1 Linear Model – LASSO Regressor

LASSO regression (Least Absolute Shrinkage and Selection Operator) diverges from standard linear regression by incorporating bias into the model. This is achieved by penalising the magnitude of the coefficients associated with the less significant features using L1 regulation and shrinking them. Consequently, this process inherently facilitates feature selection, focusing the model's attention on the most influential predictors for the target.

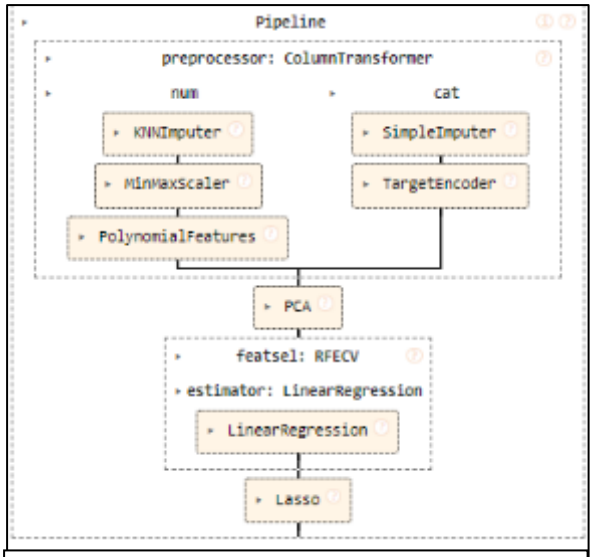

Figure 6 – Data Pipeline For LASSO Regressor

The inclusion of polynomial and interaction features enhanced model performance compared to the previous tests without these features, making their selection logical. Further attempts to improve the model were performed; PCA was tested, and it was found that the optimum PCA=15 components yielded the best performance, determined iteratively. Finally, the application of the three feature selection methods outlined in *3.0 Feature selection* were trailed, and among these it was discovered that RCEVF bolstered the performance of the model. Subsequently, the combination of the two methods, PCA=15 and RFECV yielded the best performing model. While RFECV didn't optimise the model by eliminating features as 15 remained after its implementation, the use of a linear regressor as the feature selection estimator likely resulted in the assignment of different weights to features based on their importance thus improving the overall predictive power of the model. ***Figure 6*** shows the final pipeline used for the LASSO regressor.

4.2 Random Forest Regressor

A Random Forest Regressor is an ensemble model consisting of a specified number of decision trees (n_estimators), designed to enhance generalization and mitigate the risk of overfitting compared to an individual tree. By combining predictions from multiple trees through averaging, Random Forests increase the likelihood of achieving greater accuracy, leveraging the collective wisdom of the ensemble to produce more reliable and robust predictions. In order to optimise model performance, it was necessary to perform a grid search.

rank_test_	mean_test_score	mean_train_score	max_depth	min_samples_leaf	n_estimators
1	-2140.42688	-1731.758265	18	5	140
2	-2143.473045	-1721.885114	25	5	130
3	-2144.883859	-1715.974335	25	5	170
4	-2145.385209	-1720.963671	19	5	110
5	-2145.659403	-1723.383553	23	5	120

Figure 7 – Results of Grid Search for Random Forest Regressor.

Initially, an extensive range of hyperparameters was explored to ascertain the optimal configuration for the RandomForest model. Following this exploration, refinement of key hyperparameters, notably "min_samples_leaf" and "max_depth," ensued to ensure they fell within the range conducive to optimal performance, and the grid search process was repeated. The optimal performing model was identified with a maximum depth of 18 and a RandomForest comprising 140 individual trees generated from bootstrap-aggregated data (bagging). Bagging utilizes random samples with replacement to train the decision trees, diversifying the subsets of data for each tree with the aim to reduce overfitting and improve generalization.

Deep trees with a depth of 18 indicate the presence of more complex relationships and significant feature interaction, and increasing the maximum depth beyond this threshold yielded results that were indeed sufficiently comparable for consideration. However, while increasing depth may potentially improve model accuracy depending on the specific subset of unseen data, this led to an increase in overfitting, as evidenced by the disparity in performance ratios between the test and training scores. With regards to the number of estimators, 140 proved optimum, striking a balance between ensuring the model was sufficiently complex to capture the underlying patterns in the data without becoming overly sensitive to noise or specificities of the training set.

Once optimum hyperparameters were determined, a similar fine-tuning process as described for the LASSO regressor was applied to the Random Forest model. The best-performing Random Forest model was achieved without the introduction of polynomial features attributed to the inherent feature interaction elements and the model's ability to capture non-linear relationships) or PCA, but instead (RFECV) with a linear regressor as the

estimator was utilized with the base selected features. Similarly to LASSO, model performance was not improved via feature elimination but due to the assignment of different weights for the features via the linear regressor as feature selection estimator.

4.3 Boosted Tree - XGBoost

XGBoost (Extreme Gradient Boosting) is a machine learning algorithm that integrates elements from both LASSO and Random Forest, making it a powerful tool for predictive modelling. Unlike Random Forest, where trees are built independently, XGBoost constructs decision trees sequentially, with each subsequent tree aiming to rectify the mistakes made by the previous ones. This iterative training process, known as gradient boosting, enables XGBoost to discern intricate patterns within the data by continually refining its predictions based on the errors observed in preceding iterations. In addition, XGBoost utilizes L1 and L2 regularization to avoid overfitting. By doing so, XGBoost controls the complexity of the model, enhancing the model's ability to generalize to new ***Figure 8*** shows the results of the grid search.

rank_test_score	mean_test_score	mean_train_score	max_depth	n_estimators	learning_rate	min_child_weight	reg_lambda	reg_alpha
1	-2050.991841	-1497.016808	9	175	0.1	1	1	0.1
2	-2053.557297	-1441.392087	10	200	0.1	5	1	1
3	-2057.899986	-1516.482538	9	175	0.1	3	1	0.1
4	-2061.195672	-1582.19508	9	150	0.1	5	1	0.1
5	-2061.545332	-1532.799803	9	150	0.1	1	1	0.1

Figure 8 - Results of Grid Search for XGBoost Regressor.

The optimal performance for XGBoost was achieved using a pipeline that incorporated principal component analysis with 10 components, without the inclusion of polynomial features. This was determined to be the optimum model as the gradient boosting mechanisms inherent in XGBoost, with increasing iterations, naturally capture complex feature interactions and non-linear relationships without the need for explicit polynomial features.

4.4 Ensemble - Voting regressor

A hard-voting VotingRegressor is an ensemble algorithm that combines multiple regression models, in this instance the XGBoost and Random Forest, that have been both trained on and produce predictions (votes) individually, with the final prediction of the VotingRegressor aggregating these individual predictions through averaging, resulting in a more robust and refined outcome. The predictions generated are of particular interest because outliers in the VotingRegressor are likely to have been significantly mis-predicted by both models independently, highlighting instances where the models collectively struggle to accurately predict. This insight will help identify scenarios where the model breaks down, informing strategies to enhance future model performance. This also holds true for the opposite scenario where prices are very closely predicted.

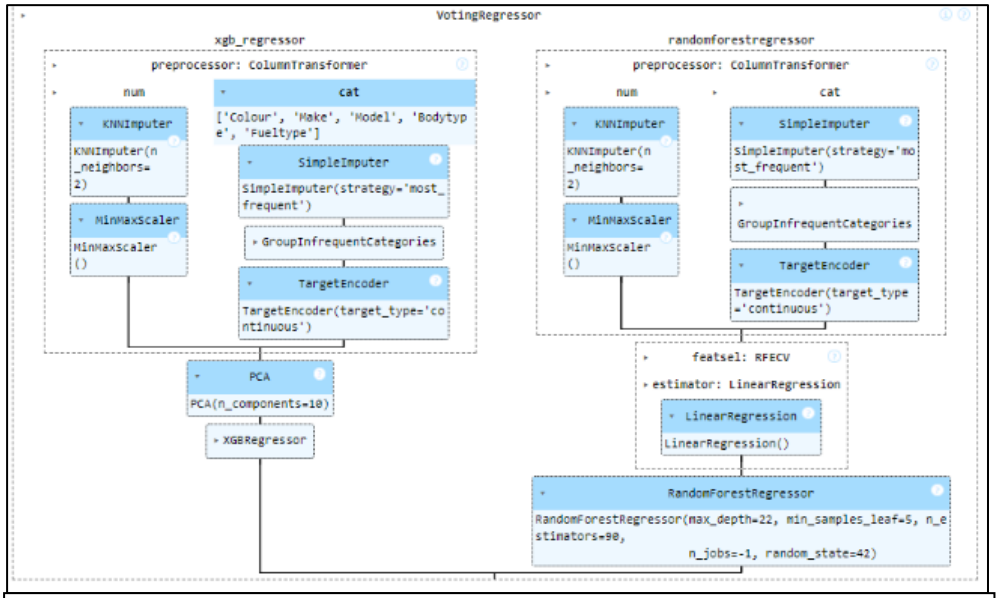

Figure 9 - Voting Regressor Pipeline, Comprised of The Individual XGB and Randomforest Pipelines Used For The Purposes Of This Investigation

The decision to exclude the LASSO Regressor from the ensemble model was based on the decrease

in performance observed with its inclusion (refer to Section 5: Model Evaluation and Analysis for further details). While considerations regarding model interactions were taken into account, given the averaging nature of the VotingRegressor, conducting a grid search to optimize hyperparameters on this ensemble was considered redundant. Instead, hyperparameters were optimized independently for the individual models that comprise the VotingRegressor as detailed previously, and those optimized hyperparameters were selected for investigation. Curious to confirm these logical assumptions, the code required to perform and execute a grid search was produced, however the sheer computational demands due to the large number of hyperparameters further confirmed this to be excessive.

5.0 Model Evaluation and Analysis

5.1 Overall Performance with Cross-Validation

Model	Mean Train MAE (£)	Mean Train R^2	Mean Test MAE (£)	Mean Test R^2
VotingRegressor	1431.25	0.95	1615.66	0.94
Random Forest Regressor	1401.12	0.95	1638.12	0.94
XGBoost Regressor	1533.97	0.95	1674.83	0.94
LASSO Regressor	3191.13	0.81	3199.89	0.81

Figure 10 - Mean Cross Validated Scores For Respective Models.

The VotingRegressor emerged as the best performing model overall, with a train Mean Absolute Error (MAE) of £1431.25 and a test MAE of £1615.66. It also demonstrated exceptional capability in accounting for variance and capturing the underlying patterns within the dataset, as evidenced by the R^2 scores of 0.95 and 0.94 for train and test data respectively, suggesting a robust understanding of the linear and non-linear relationships in the data. Both scoring metrics strike a delicate balance between overfitting and underfitting, with train scores slightly outperforming test scores as expected. This disparity ensures the model avoids both underfitting and overfitting, as excessively superior performance for train would indicate. By maintaining this equilibrium, admirable model performance is achieved without overly tailoring its predictions to the idiosyncrasies of the training set. This balance instils confidence in the model's ability to generalize well to unseen data and considering the wide price range of cars included in the investigation (£1695.0-£59995.0), the predictions generated by this model hold considerable weight and reliability.

The Random Forest regressor emerged as the next best-performing model, exhibiting similar levels of variance capture as the ensemble model with R^2 scores of 0.95/0.94 for train and test, and MAE values of £1401.12 and £1638.12 for train and test data, respectively. Although an increased degree of overfitting is observed in terms of MAE in comparison with the other models, this can be attributed to the inherent architecture of the Random Forest regressor, being complex and therefore increasing the chance of fitting too closely to the training data. Nevertheless, a reasonable balance was still established.

XGBoost followed as the third best-performing model, showcasing the most balanced model with the smallest discrepancies between train and test MAE (£1533.97 and £1674.83, respectively), along with a competitive capture of the variance (0.95 and 0.94 R^2 for train and test). The top three models displayed similar performance across all metrics, suggesting a robust and consistent predictive capability. The LASSO Regressor however was the worst performing of the models, capturing the least amount of variance (train/test) both had an R^2 of 0.81 and also had the most inaccurate average predictions, with £3191.13 train MAE and £3199.89 test MAE respectively. While the introduction of polynomial features, along with the regulatory hyperparameters of the LASSO regressor improved performance compared to the basic linear regressor used in the first iteration, this was not significant enough to be competitive with the other, more complex models.

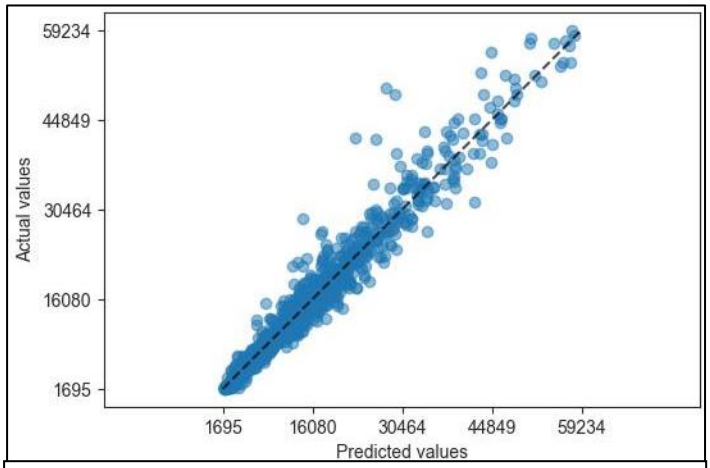

Figure 11- True vs Predicted Analysis plot for Voting Regressor

5.2 True vs Predicted Analysis-

Figure 11 shows the resultant scatter graph of predicted vs actual values produced by the VotingRegressor. There is a high density of data points surrounding the identity line at lower values, indicating the model has successfully learned the patterns of the data for cars priced in this region. As car price increases, it appears the likelihood for predicted prices to deviate further from the actual increases, indicated by the increasing presence of outliers and therefore larger variability in predictions. This suggests that the model generally works better for cheaper cars and model performance decreases with increasing price, which will have a negative impact on the model's overall performance. This change in trend becomes apparent for cars priced around £30,464.

While this ensemble model does produce fewer outliers compared to the other models, there are still instances where the average prediction deviates by more than £10,000 from the true value. This indicates notable mispredictions occurring for these instances across both models in the ensemble. Further investigation led to another iteration involving cars priced from £1695 to £30,000 on the VotingRegressor. This yielded training and test scores of 0.95 and 0.94, respectively, with corresponding MAE of £1616.84 and £1428.58(~£1 off on average in comparison with full range). This suggests that while significant outliers exist above £30,464, the overall model performance does not notably decrease beyond this threshold. In fact, it indicates that, excluding the extreme outliers in predictions, the model's accuracy above £30,000 may actually be superior to that below. However, this assertion cannot be definitively confirmed or denied without further refinement of the model to address underlying reasons for the significant mispredictions.

5.3 Global and Local Explanations with SHAP

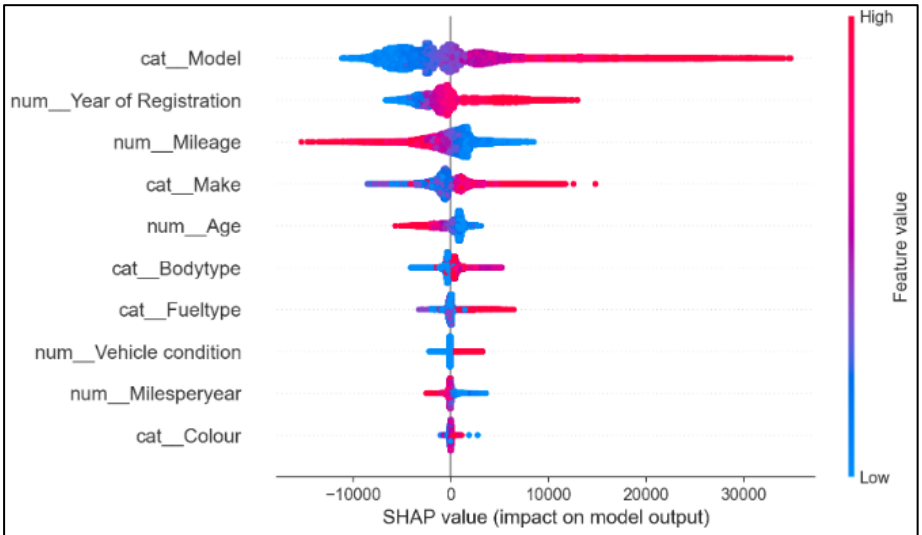

Figure 12 – Global SHAP Graph Produced From Random Forest Regressor Ordered By Feature Importance (Ascending)

From ***Figure 12***, model is determined to be the most significant feature by SHAP, and influence on price predictions for car models deemed to be expensive(high feature value) can stem upward from around £8000-£30,000, indicated by the red shade on the line. Models that are on average more expensive correspond to larger price predictions overall and this correlation is unsurprising, considering that higher model prices typically align with higher overall prices and the model is the most unique physically identifying feature present in the dataset. The increased density observed around the £0 mark likely stems from several factors. As average model price decreases, the possible range of influence on price predictions also decreases, evidenced by the length of red against other colours (higher feature value to low) that comprise the SHAP line for model.

The relationship between average model price and its impact on the model output does not exhibit a symmetrical pattern for positive and negative impacts. Specifically, the range for possible prices predictions for more cheaper models is not as wide as that for more expensive models. This could be attributed to the observed range of cars in the dataset, which spans from £1695.0 to £59995.0, with fewer instances observed at higher prices compared to lower ones. If the dataset had a more even distribution among prices, one would anticipate the appearance of the SHAP line to be more symmetrical. A similar trend is observed with feature make, and while its price influence range isn't as wide in general as for model the line appears more symmetrical. This indicates a clear, consistent distinction amongst the categories within make, with cars makes being perceived to be more prestigious being more expensive and vice versa, with the influence on price prediction being relatively linear corresponding to the market category of the make. Potentially if it were more widely known that manufacturers share parts (for example, sports car Pagani Zonda using the same engine as pedestrian Mercedes-Benz S600) this observed trend may be less prevalent.

With increasing mileage, impact on the model output decreases. The effect of higher mileage is more pronounced on price prediction for reducing price, than with low mileage is for increasing, exhibiting non-symmetry as was the case for model. Car parts are mechanical so the further the distance travelled, the greater the likelihood of the depreciation of the moving parts required. Albeit wear and tear can be dependent on how carefully the vehicle has been driven and its upkeep, but collecting the information as to the main reason the car was utilised to provide further information towards this would be a useful metric to capture. However, because of the sheer number of components involved in making a car run, the vast majority will eventually become uneconomical to repair with increasing mileage. Depreciation of parts also leads to a reduction in performance, further explaining the observed trend.

Age exhibits a similar trend, albeit with less overall influence and less pronounced effects and the weaker influence of age can be attributed to several factors. While unlikely to constitute a large proportion of instances due to cut off date being 1986, vintage cars may increase in price with age, potentially counteracting the general trend of depreciation. Secondly, the presence of newer cars that may be cheaper than older ones can also dampen the overall impact of age on the target price. A similar explanation can be provided for Milesperyear, but to an even lesser extent. While its true that cars that are driven at a greater intensity are generally cheaper than those driven with less intensity (confirmed by correlation coefficient of -0.21), this effect is less significant when compared to the impact of age and mileage likely due to the offset from various other factors such as maintenance history, overall condition, and market demand.

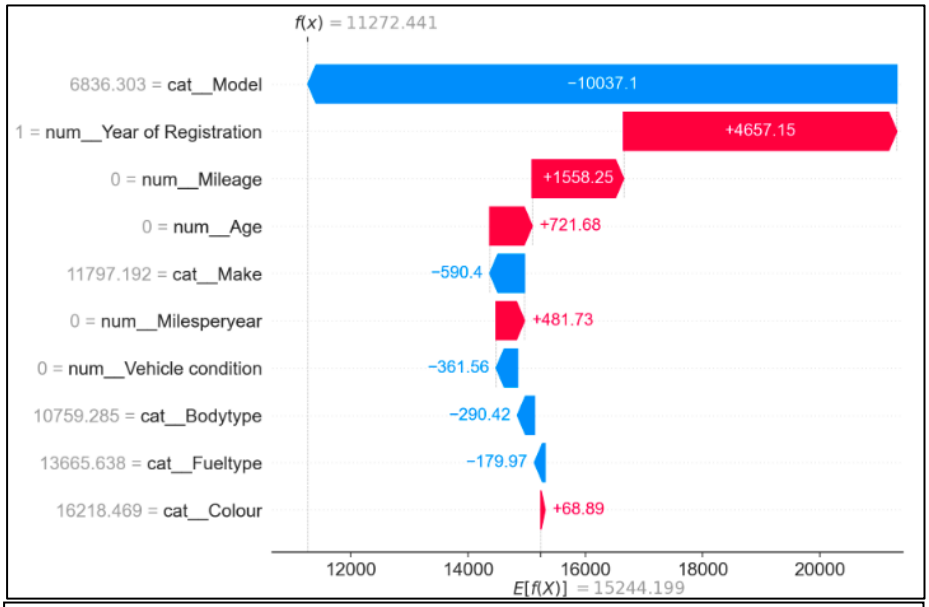

Figure 13 - Shap Plot For individual Instances, Where f(x) is the Random Forest Regressors Price Prediction and E[f(x)] Is The Average Expected Across All Instances.

In this instance, the most significant feature impacting the model's prediction is the car's model which is in line with global SHAP predictions. Its contribution substantially lowers the predicted price (-£10037.1) from the average expected price of £15244.19, indicating that this particular car model is, on average, associated with lower prices with respect to the overall distribution of the target. Furthermore, the car's year of registration, assumed to be recent, significantly increases the predicted price by £4657.15. This suggests that the car is relatively new, further supported by the

increase in price due to mileage indicating it hasn't been driven much, along with the price increase due to its age and Milesperyear. Interestingly, while Milesperyear is considered the second least important feature overall, it holds more significance for this instance, appearing as the fifth most important feature and contributing to a price increase of £481. This implies that despite being pre-owned (as inferred from the decrease in price prediction of £361.56 based on vehicle condition), the total mileage and the utility of the vehicle outweigh the impact of ownership classification.

All machine learning algorithms included in this report are expected to face challenges when encountering instances where the model is unseen within the training data but the make is present, especially if it involves a cheaper make producing a more expensive model, such as sports cars, or vice versa with more expensive makes producing cheaper models. This complexity arises because while certain factors like body type (e.g. SUV) and low mileage may help mitigate the price disparity, the dataset lacks sufficient features to capture all the intricacies involved. While custom class "Groupinfrequentcategories" was introduced to try and negate this and does to a certain extent, the problem still remains. To tackle this issue, Autotrader should consider incorporating additional features such as MPG (miles per gallon) and engine size. Doing so would undoubtedly enhance the model's performance and predictive accuracy.

5.4 Partial Dependence

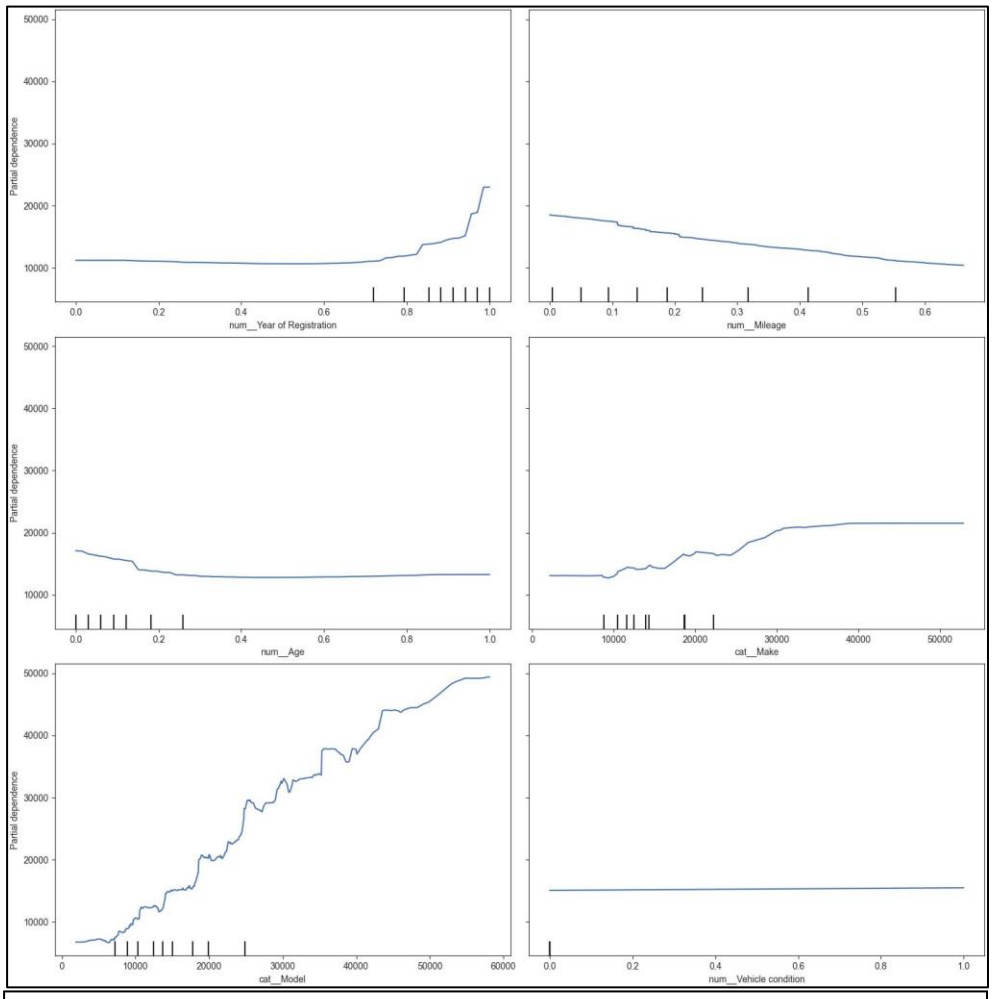

Figure 14 - Partial Dependency Plots Obtained from Random Forest Model. (Colour, BodyType, FuelType and Milesperyear not included)

In *5.3 Global and Local Explanations with SHAP,* model is identified as the most influential feature for predictions, with make ranking fourth. In ***Figure 14***, the partial dependency plot of these two features with price reveals considerable non-linearity, particularly pronounced for model. Notably, this elucidates why linear models struggle to perform as well as other, more complex models as they fail to capture the intricate non-linear relationships between features and target, especially concerning the most influential feature in the dataset. In the case of model, there appears to be a predominantly positive, generally linear relationship observed until the price reaches around £7000. Beyond this threshold however, non-linearity becomes more prominent. This transition suggests that while lower-priced models may exhibit a more straightforward relationship with price, higher-priced models introduce complexities that will prove difficult to be captured by linear algorithms. Furthermore, the periods of linearity followed by

non-linearity may indicate that the relationship between the model and make with price varies across different segments and subgroups of the data. Therefore, specific combinations of model or make, in conjunction with other features can instigate non-linear fluctuations in the price of a vehicle. These fluctuations may stem from various factors like brand popularity or prevailing market trends leading to deviations from the anticipated linear relationship between individual features and price. For example, a blue Mercedes may command a higher price than a red one, while the inverse may be true for BMWs within the same price brackets. This provides partial explanation as to the improvement of the linear regressor for the second iteration, attributed to the incorporation of combination and polynomial features, while also providing justification for its lack of competitiveness with models that inherently capture these relationships without requiring artificial feature engineering.

A negative linear relationship between mileage and price is observed, further supporting the explanation provided in section *5.3 SHAP.* Additionally, slight fluctuations are noted near the 0.1 and 0.2 percentiles of mileage, suggesting that vehicles falling within this range are sold at an even higher price as they are perceived as fresher with fewer wear and tear issues, resulting in higher demand. While the trend is expected to somewhat continue for the entire range of mileage, the relationship appears to plateau around the 0.7 percentile. This indicates that cars with mileage above this threshold are classified similarly by buyers, being cars that have been extensively driven. Therefore, regardless of the specific value on the odometer, mileage above this threshold has less impact on price. This trend also holds true for age but, but the relationship diminishes past the 0.3 percentile. Multiple sharp peaks are evident in the partial dependency plot as the year of registration approaches the most recent. These peaks suggest sudden increases in price for cars registered in recent years. This can be attributed to the included technological advancements in newer car models, as well as the fact that a new car loses value as soon as it is purchased and driven off the forecourt and by the end of the first year although varied, can lose around 40% of its value.

For the partial dependency plot for vehicle condition, a positive, linear trend would be expected, indicating as price increases so does vehicle condition, however a flat line was depicted by the plot. The absence of slope indicates the absence of any discernible relationship between a car's vehicle condition and the resultant price. However, previously engineered features, although not utilised for the machine learning investigation due to concerns regarding data leakage offer valuable insights into the dataset's dynamics. Specifically, the disparity between the respective correlation coefficients of average model type new and used (0.74 vs 0.58 respectively) suggests that new cars have a stronger relationship with price than older. The inability to capture the relationship on a PDP plot can therefore be attributed to the strength of the overall correlation of vehicle condition with price, exhibiting a weakly positive correlation coefficient of 0.4. Furthermore, the disparity between distribution of new and used cars in the data set (92.2% used vs 7.8% new) also means that the full effect of vehicle condition isn't as understood as other features. While a positive, linear relationship might be expected within unique models for the partial dependence of vehicle condition, the weakness of the overall correlation prevents this relationship from being adequately captured. This explanation holds true also for milesperyear which saw no discernible relationship. The justification for the other features that also failed to produce a trend line of value (colour, fuel type, body type) can be attributed to their non-ordinal nature.

Name – Omari Ricketts

Student ID - 21411371